Kopilotti Sales - product overview

Kopilotti Sales is a digital car salesperson and a sales channel for dealerships. It negotiates and accepts a price on the dealer's behalf, within the authority and limits set by the dealer.

1. Product purpose

Vehicle information and other purchasing steps can be online while price negotiation still requires a manual exchange. Kopilotti Sales performs the selling task under the dealer's authority and takes the customer toward an agreed deal.

The dealer defines the policy. The system applies it consistently. Cases outside the automated boundary are escalated for human review.

2. Customer journey and responsibilities

Vehicle listing / list-price purchase or price negotiation / agreed deal and price / sales contract / electronic signature / payment or financing.

An agreed deal means that the vehicle and price have been agreed under the dealer's authority. The salesperson does not need to restart the same price negotiation. The contract, electronic signature and payment or financing remain separate steps; the vehicle is not yet paid for or handed over. Price validity and availability are checked under the agreed terms.

Sales agrees the price under the dealer's authority. The dealer's systems and selected providers handle the contract, electronic signature and payment or financing. Funds go directly to the selling dealership. People handle exceptions. The language model does not decide the price.

The public demo is a non-binding demonstration. List-price and negotiated-price paths are implemented, but the current flow stops before real payment or financing. The new atomic deal-record, reservation and inventory-lock path is implemented and tested separately and is not connected to the current customer journeys. Rollout requires verified availability, data handoff, contract, electronic-signature and payment or financing connections.

3. Commercial boundaries

The dealer defines the commercial policy, including the price floor, target price, list price, round limits, and escalation rules. Kopilotti Sales applies the policy; it does not invent a pricing strategy.

A stale policy, missing vehicle, mismatch between the vehicle and policy, or policy conflict leads to escalation instead of a guessed decision. A person retains authority over exceptional cases.

4. Deterministic policy engine

The policy engine returns one of four states: accept, counter, reject, or escalate. The price floor bounds acceptance and counteroffers. The same validated input and policy produce the same commercial decision.

The commercial decision is made by a server-side policy engine isolated from the LLM. This boundary reduces the impact of LLM manipulation but is not presented as complete protection against every attack.

5. Reservation and double-booking prevention

The PostgreSQL model permits only one active reservation for the same dealer and vehicle through a database-enforced unique index. The accepted decision, reservation, session transition, and audit event share a transaction. If one step fails, the unit of work is rolled back.

Reservation mechanisms are implemented and tested but have not been production-verified in this review.

6. Persistence and application audit history

The published demo's production backend stores negotiation sessions durably in PostgreSQL. Session access is tenant-scoped at the application boundary, and tenant-relationship integrity is enforced by database constraints.

Persistence paths for purchase sessions, idempotency records, decisions, and audit events, together with the application audit history and append-only application path, are implemented and tested but remain outside this limited production-verification scope.

This describes application and database behavior in the repository. It does not claim an immutable external ledger, event-sourced architecture, or production verification.

7. DDN and verifiability

DDN (Deterministic Decision Network) preparation includes deterministic canonicalization, domain-separated hashes, a bounded submission adapter, and a safe local boundary for constructing and displaying a receipt link for a locally verified result.

A hash supports integrity checking, but a hash alone does not authenticate the receipt issuer.

Live DDN verification, signer authentication, quorum and trust profiles, public anchoring, and an independent offline verifier are roadmap items. The current implementation does not present them as complete production capabilities.

8. Security boundaries

- The LLM may support conversation but does not decide the price or commercial outcome.
- LLM isolation reduces the impact of manipulation but is not a promise of complete prompt-injection protection.
- A hash shows integrity-related consistency, not issuer identity or a trust chain.
- Roadmap capabilities are not prerequisites for the current decision flow.
- This overview does not authorize deployment, migrations, or access to production services.

9. Production-readiness status

Status: **limited production verification completed on 26 August 2026**.

The published demo's production backend was verified for durable PostgreSQL storage of negotiation sessions, tenant-scoped session access, and database-enforced integrity for tenant relationships. A fresh backup-and-restore test and the schema and readiness gates passed before release.

This is a limited production verification, not a general production-ready claim for the entire product. Reservation and audit mechanisms, the payment-capable transaction path, live DDN verification, and a production VIS connection retain the boundaries described below.

The publicly documented current scope and production boundaries are available in the Kopilotti Sales public README.

10. Production-verified scope - 26 August 2026

- durable PostgreSQL storage of negotiation sessions in production
- tenant-scoped negotiation-session access
- database-enforced integrity for tenant relationships
- production schema and readiness gates
- backup-and-restore test passed before release

The verification applies only to the limited production scope listed above.

11. Implemented and tested

- digital price negotiation
- a server-side commercial decision isolated from the LLM
- deterministic `ACCEPT`, `COUNTER`, `REJECT`, and `ESCALATE` logic
- dealer-defined commercial boundaries and price-floor enforcement
- deterministic canonicalization and hash generation
- a safe local boundary for constructing and displaying a receipt link
- an authenticated, dealership-scoped, read-only negotiation summary

12. Not production-verified

- atomic vehicle reservation
- database-enforced prevention of concurrent active reservations
- application audit history and hash chain
- append-only application path for audit events
- the dealer negotiation summary API

Reservation and audit mechanisms are implemented and tested but have not been production-verified in this review.

13. Roadmap and research directions

- live DDN verification
- signer authentication and pinned trust profiles
- quorum verification and public anchoring
- an independent offline verifier
- byte-exact runtime replay
- a signed committed decision artifact
- proof-gated execution
- zero-knowledge proofs
- approved RTO/RPO targets
- named operational owners and response times
- an Admin interface for agreed deals
- a consent-based customer-contact handoff to the dealer
- an email notification to the dealer
- payment or financing features
- a separate staging environment

These are target or research directions, not current capabilities.

14. Demo and further information

- Open the demo
- Suomenkielinen tuote-esittely
- Suomenkielinen A4-PDF
- Repository README
- License

The demo illustrates the product flow. It does not prove production verification for capabilities marked as not production-verified.